DOCKER FUNDAMENTALS
FOR AGENTIC AI

Step-by-Step Guide:
Containerizing our FastAPI Agent Validation Platform

What you will learn

- Containerize our FastAPI validation app
- Build a Docker image
- Run the container
- Verify the API & tests
- View container logs
- Stop and remove the container

Gouvernance des Agents IA • v1.0.0
Auteurs : Khaoula Adeli & Khalil Hassani Khalfaoui

2. Our FastAPI Validation App

We will containerize our FastAPI validation and compliance platform. The platform executes tests and evaluates AI Agent behavior in an offline, repeatable environment.

Key Features of the App:

• **FastAPI Server**: Exposes endpoints for executing tests (`/api/tests/run`) and exporting the workflow configuration.
• **Test Engine**: Executes 20 test cases in a Golden Set and calculates intent correctness, quality, and tone scores.
• **Frontend Dashboard**: Displays metrics, compliance reports, and an interactive playground.

Project Structure:

```
gouvernance-agents-ia/
|-- backend/
|   |-- app/
|   |   |-- main.py         # FastAPI Server
|   |   |-- services/
|   |   |   `-- agent_service.py # Simulated Agent
|   |   |-- tests_engine/
|   |   |   `-- engine.py   # Test Orchestrator
|   |   `-- validators/
|   |       `-- conformity.py # Rules Engine
|   `-- requirements.txt    # Python Deps
|-- frontend/               # Dashboard static files
|   |-- index.html
|   `-- app.js
`-- backend/Dockerfile      # Docker Configuration
```

3. requirements.txt

Python dependencies are listed in `requirements.txt`. These libraries are installed inside the Docker image during the build process to guarantee environment isolation.

Core Dependencies Explained:

- **fastapi**: The core web framework utilized to build the API.
- **uvicorn**: An ASGI web server implementation used to run the FastAPI app.
- **pydantic**: Used for data validation and schema definitions.
- **pytest**: Test framework to launch and validate the execution suite.

requirements.txt (File content):

```
fastapi>=0.110.0
uvicorn>=0.28.0
pydantic>=2.0
pytest>=8.0
anyio>=4.0
```

4. Dockerfile

A **Dockerfile** contains the sequence of instructions Docker uses to build the container image.

Instruction	Purpose in our project
FROM	Uses the official lightweight Python 3.12 image as a base.
WORKDIR	Sets the directory inside the container to `/app`.
COPY	Copies files from the host into the container filesystem.
RUN	Installs Python libraries listed in `requirements.txt`.
EXPOSE	Documents that port 8000 is open for incoming traffic.
CMD	Defines the command that launches the server via Uvicorn.

Dockerfile Configuration:

```
FROM python:3.12-slim
WORKDIR /app

COPY backend/requirements.txt ./backend/requirements.txt
RUN pip install --no-cache-dir -r ./backend/requirements.txt

COPY backend ./backend
COPY frontend ./frontend

EXPOSE 8000
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

CMD ["python", "-m", "uvicorn", "backend.app.main:app", \
     "--host", "0.0.0.0", "--port", "8000"]
```

5. Open Terminal and Navigate to Project

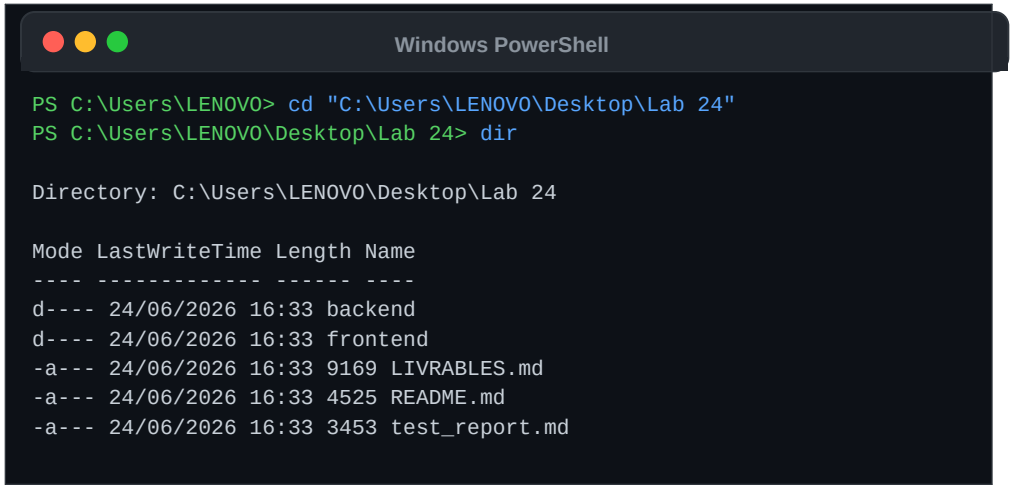
To containerize the application, start by opening your terminal or PowerShell console and navigate to the project directory.

Commands used:

• **cd**: Change directory to the root of the project.

• **dir** (or **ls** on Unix): Lists files in the current folder. Verify that the `backend/` and `frontend/` folders are visible.

Console Output:



6. Build the Docker Image

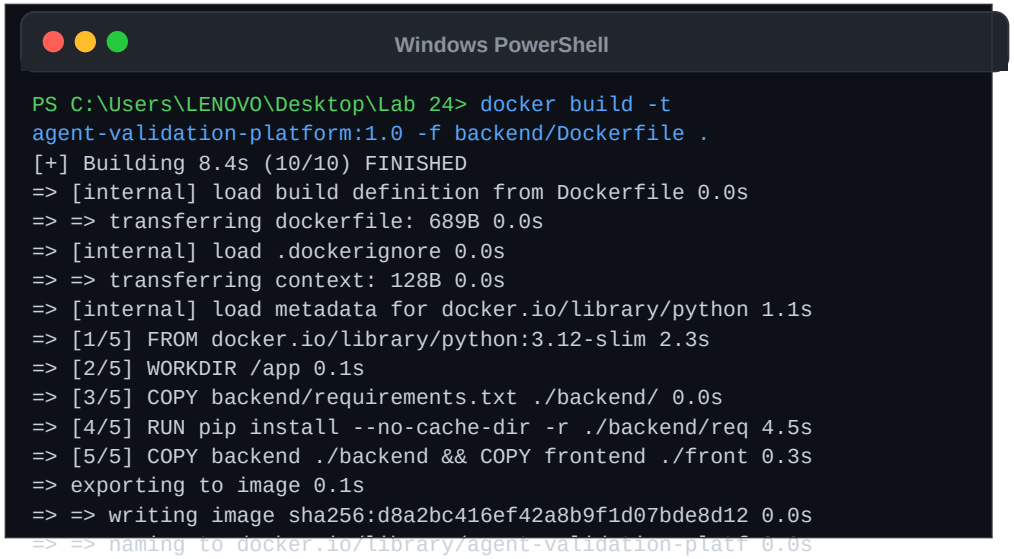
Build the Docker container image from the Dockerfile. This bundles Python, our dependencies, and all source code into a single immutable artifact.

Command Syntax:

```
docker build -t name:tag -f Dockerfile_path context
```

- **-t agent-validation-platform:1.0**: Tags the image with a name and version.
- **-f backend/Dockerfile**: Specifies the path to the Dockerfile.
- **.**: Context is the current folder (all files are copied relative to this root).

Docker Build Process:



7. Verify the Image

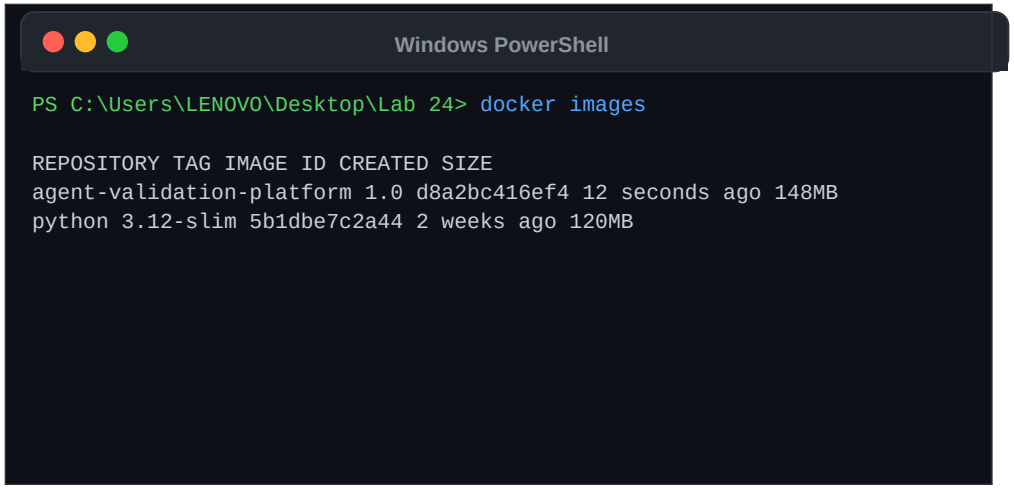
Verify that the image was built successfully and is registered in Docker's local image cache.

Command used:

• **docker images**: Lists all locally cached Docker images, including their repository name, tag, image ID, creation date, and size.

The `agent-validation-platform` image is self-contained and weighs approximately 148MB (which includes the entire Python runtime, requirements, and frontend/backend files).

Docker Local Cache:



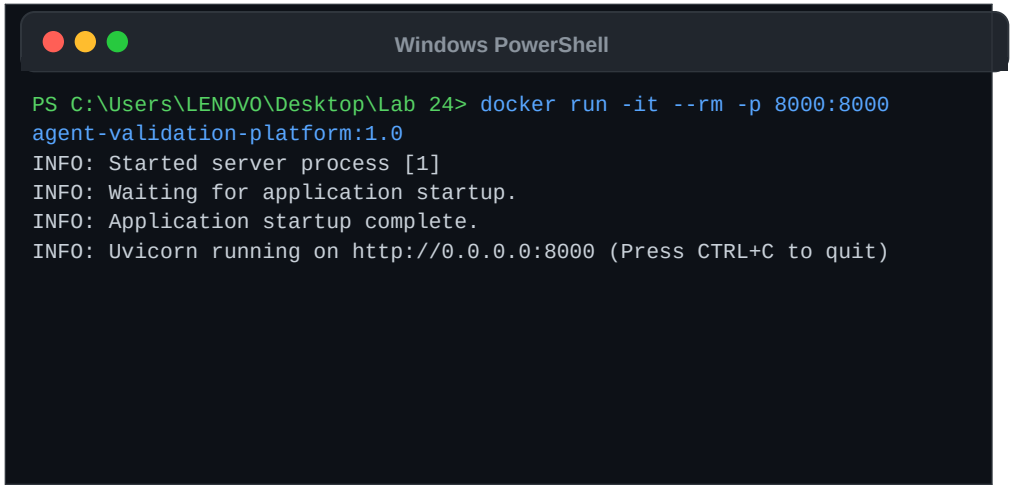
8. Run the Container

Instantiate and run the Docker container. This starts the FastAPI web server, mapping the ports to allow access from the host machine.

Command details:

• **-it**: Interactive mode with terminal output feedback.
• **--rm**: Clean up and delete container files when stopped.
• **-p 8000:8000**: Map port 8000 on the host to port 8000 inside the container. This makes the dashboard accessible at `http://127.0.0.1:8000`.

Container Startup logs:



9. Test the Agent & API

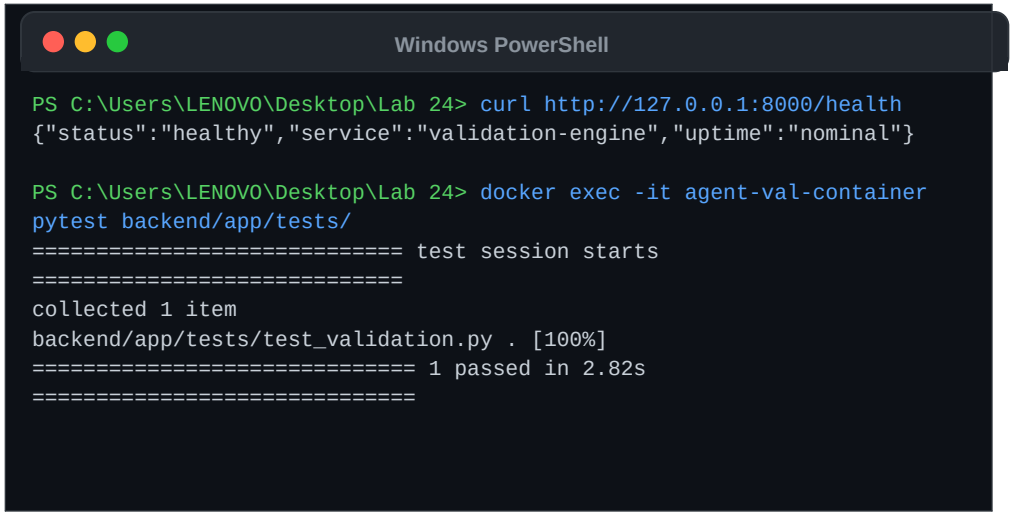
Validate the running API container. We can query the health endpoint and execute the pytest suite inside the active container environment.

Validation methods:

• **curl**: Verify that the FastAPI backend is running and healthy on port 8000.

• **docker exec**: Run commands inside an already running container. We use it to trigger our 20 business compliance test cases.

Testing & Verification:



10. View Logs (Detached Mode)

Run the container in detached background mode. This is the preferred way to run services in staging or production environments.

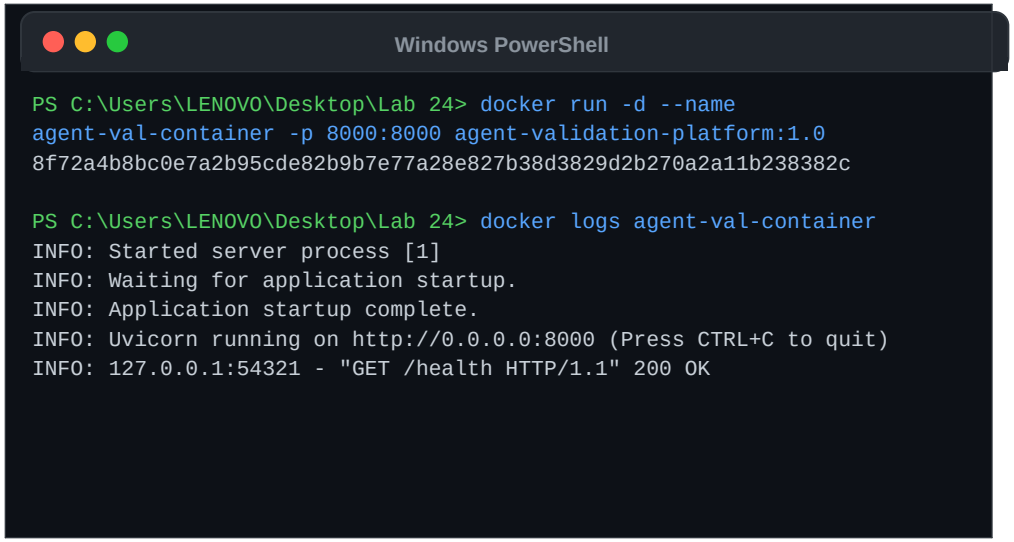
Detached command flags:

• **-d**: Detach and run the container in the background. The terminal immediately prints the container ID.

• **--name agent-val-container**: Assign a human-readable name to the container to manage it easily.

• **docker logs name**: Retrieve and view the standard output/error logs of the background container.

Background Execution Logs:



11. Stop the Container

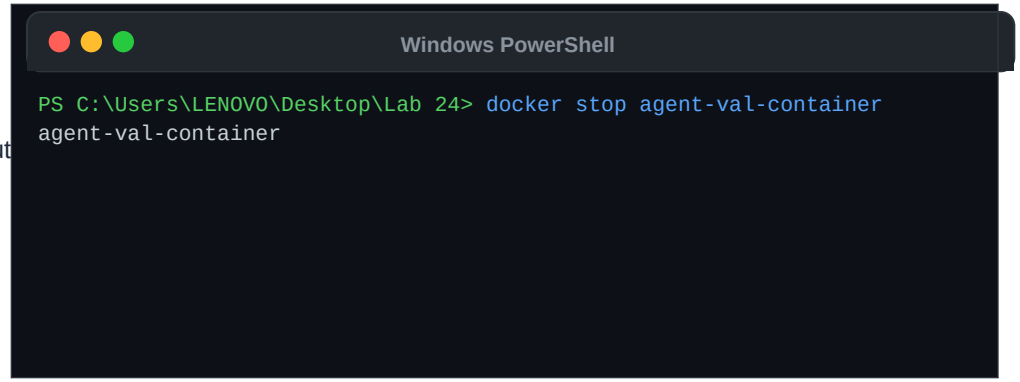
Stop the running background container. This gracefully terminates Uvicorn and FastAPI before halting the container instance.

Command details:

• **docker stop agent-val-container**: Sends a SIGTERM signal to Uvicorn (PID 1), allowing open connections to close properly, followed by a SIGKILL if it doesn't shut down in 10 seconds.

Stopping a container does not delete its filesystem, meaning it can be restarted later.

Graceful Stop:



12. Remove the Container

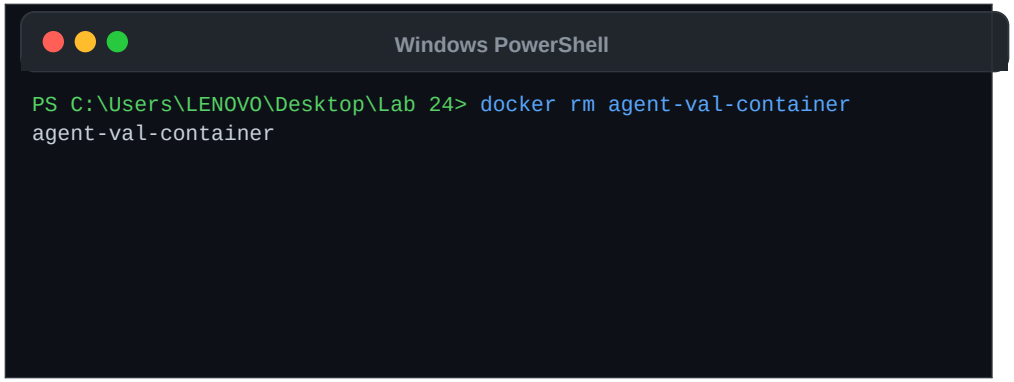
Delete the stopped container from the local storage. This cleans up container metadata and releases the assigned port mapping and container name.

Command details:

• **docker rm agent-val-container**: Deletes the container instance. All temporary data inside the container is permanently erased.

Once removed, the port 8000 is released and you can launch a new container instance with the name `agent-val-container`.

Remove Container:



13. Summary

You have successfully containerized and validated the FastAPI Agent Validation Platform using Docker!

What was covered:

✓ Verified project structure and code dependencies.
✓ Explanatory breakdown of Dockerfile instructions.
✓ Navigated files and built the Docker image.
✓ Ran the container, mapping port 8000.
✓ Verified API health check and tests inside the container.
✓ Launched in detached mode and retrieved container logs.
✓ Gracefully stopped and deleted the container.

Validation Status

IMAGE STABLE

- Repository: agent-validation-platform
- Tag: 1.0
- Base Runtime: Python 3.12-slim
- Exposed Ports: 8000
- Test Suite execution inside Docker: **100% OK**